

动态规划与动态规划常见优化方法

Nesraychan

Tsinghua University

July 24, 2026

状压 dp

一般的动态规划，如序列 dp，区间 dp，他们的状态都很显然，要么是序列上的一个位置，要么是一个区间。而有些问题你需要记录的东西则很多，如 $\{0, 1, \dots, n\}$ 中的数有哪些出现了。

状压 dp

一般的动态规划，如序列 dp，区间 dp，他们的状态都很显然，要么是序列上的一个位置，要么是一个区间。而有些问题你需要记录的东西则很多，如 $\{0, 1, \dots, n\}$ 中的数有哪些出现了。

在这种时候，状态数为 2^n 的，我们肯定也不能开一个 n 维数组记录。所以我们要使用状压 dp。

所谓状压 dp，事实上就是将每个状态和一个整数构建双射，这样我们只需要记录这个数是什么，并构建两侧的映射，便可以知道我们到底在什么状态了。

[NOI2015] 寿司晚宴

有 $n - 1$ 种寿司，其美味度分别为 $2, 3, \dots, n$ 。

小 G 和小 W 各自选择若干种寿司品尝，允许不选择任何寿司。每种寿司至多被其中一人选择。

若对于任意小 G 选择的美味度 x 和小 W 选择的美味度 y ，均有 $\gcd(x, y) = 1$ ，则称该选择方案是和谐的。

求和谐的有序选择方案数，并对给定的正整数 p 取模。

输入两个整数 n, p ，输出答案模 p 的结果。

$2 \leq n \leq 500, 1 \leq p \leq 10^9$ 。

[NOI2015] 寿司晚宴

这个要求，就是两个人选择的素因子集合不交。

[NOI2015] 寿司晚宴

这个要求，就是两个人选择的素因子集合不交。

一个想法是记 $f(i, S, T)$ ，表示决策了前 i 个寿司，两个人的素数集合分别是 S, T ，时候的方案数。但你容易发现，这个状态数爆炸了。

[NOI2015] 寿司晚宴

这个要求，就是两个人选择的素因子集合不交。

一个想法是记 $f(i, S, T)$ ，表示决策了前 i 个寿司，两个人的素数集合分别是 S, T ，时候的方案数。但你容易发现，这个状态数爆炸了。

优化考虑将素数分成 $\leq \sqrt{n}$ 和 $> \sqrt{n}$ 的两类，显然一个数只会有一个第二类素数。我们对素数中存在的第二类素数分类，那么就要求同一类中最多只在一边出现。

[NOI2015] 寿司晚宴

这个要求，就是两个人选择的素因子集合不交。

一个想法是记 $f(i, S, T)$ ，表示决策了前 i 个寿司，两个人的素数集合分别是 S, T ，时候的方案数。但你容易发现，这个状态数爆炸了。

优化考虑将素数分成 $\leq \sqrt{n}$ 和 $> \sqrt{n}$ 的两类，显然一个数只会有一个第二类素数。我们对素数中存在的第二类素数分类，那么就要求同一类中最多只在一边出现。

而所有第一类素数只有 8 个，那么我们只需要额外多记一个 0/1，表示当前这一类的选择情况就行了。时间复杂度 $O(2^{16}n)$ 。

[CF2062H] Galaxy Generator

平面上的星用点 (x, y) 表示。若两颗星横坐标或纵坐标相同，且它们之间没有其他星，则称它们直接相连；由直接相连关系形成的连通块称为星系。

可以反复在空点加入新星，但要求该星加入后能与至少 3 颗星直接相连。定义一个星集的价值为操作后可能得到的最少星系数。

给定 $n \times n$ 的 01 矩阵 a ，其中 $a_{x,y} = 1$ 表示 (x, y) 处有星。设所有星构成集合 S ，求 S 的所有非空子集的价值之和，对 $10^9 + 7$ 取模。

$$1 \leq n \leq 14。$$

[CF2062H] Galaxy Generator

先考虑星系数怎么算：首先对于两个连通块，如果他们对应的矩形的区域在一维有交，我们肯定将其合并。另一方面，如果现在的所有矩形区域没有交，它们肯定不能再合并。所以我们就有了一个计算的方法。

[CF2062H] Galaxy Generator

先考虑星系数怎么算：首先对于两个连通块，如果他们对应的矩形的区域在一维有交，我们肯定将其合并。另一方面，如果现在的所有矩形区域没有交，它们肯定不能再合并。所以我们就有了一个计算的方法。

那么我们现在需要计算的就有这样的两个东西： $f(i, j, l, r)$ 表示只考虑 $[i, j] \times [l, r]$ 这段区域，这些点恰好合并成整个区域的方案数。以及 $g(i, j, S)$ 表示考虑了横坐标在 $[i, j]$ 之间的星系，他们纵坐标占用的集合为 S 。

[CF2062H] Galaxy Generator

先考虑星系数怎么算：首先对于两个连通块，如果他们对应的矩形的区域在一维有交，我们肯定将其合并。另一方面，如果现在的所有矩形区域没有交，它们肯定不能再合并。所以我们就有了一个计算的方法。

那么我们现在需要计算的就有这样的两个东西： $f(i, j, l, r)$ 表示只考虑 $[i, j] \times [l, r]$ 这段区域，这些点恰好合并成整个区域的方案数。以及 $g(i, j, S)$ 表示考虑了横坐标在 $[i, j]$ 之间的星系，他们纵坐标占用的集合为 S 。

这两个东西是可以互相转移的： f 的转移只需要整体减空白，然后二维差分； g 的转移则是直接的状压 dp。于是做完了，时间复杂度 $O(n^5 2^n)$ ，常数很小，可以轻松通过。

动态规划，是什么？

讲完了状压 dp，我们不妨考虑一些更本质的东西：那就是，啥是动态规划？

动态规划，是什么？

讲完了状压 dp，我们不妨考虑一些更本质的东西：那就是，啥是动态规划？

你会发现，大部分动态规划，都是从一个状态，经过一些变换（乘一些东西，加一些东西），然后走到另一个状态，很多时候是不会有两个状态之间乘法的，换句话说，是线性的。

动态规划，是什么？

讲完了状压 dp，我们不妨考虑一些更本质的东西：那就是，啥是动态规划？

你会发现，大部分动态规划，都是从一个状态，经过一些变换（乘一些东西，加一些东西），然后走到另一个状态，很多时候是不会有两个状态之间乘法的，换句话说，是线性的。

什么是状态？状态指的可以是序列上的一个位置 i ，或者像刚刚状压 dp 一样，是一个形如 (i, S) 的东西。同时，也可以更复杂。

真的的压缩状态的动态规划

但如果你把一个状态抽象成一个点，转移抽象成一条边，你会发现，这其实就是在一张有向图上求：最短路/最长路/路径数。并且，一般来说，这张有向图都是 DAG。

真的的压缩状态的动态规划

但如果你把一个状态抽象成一个点，转移抽象成一条边，你会发现，这其实就是在一张有向图上求：最短路/最长路/路径数。并且，一般来说，这张有向图都是 DAG。

所以我们的目标就是找到所有的状态和所有的边，这样自然可以 dp。在上面的部分， S 都是一个集合，表示每个数是否存在。但事实上，这个状态可以更复杂。就像我们上面说的，我们只需要给每个状态找到一个和整数之间的双射，就可以满足我们的目标。

真的的压缩状态的动态规划

那么我们怎么找到这样的双射呢？

真的的压缩状态的动态规划

那么我们怎么找到这样的双射呢？

搜索！

真的的压缩状态的动态规划

那么我们怎么找到这样的双射呢？

搜索！

所谓状态，到底是什么？

真的的压缩状态的动态规划

那么我们怎么找到这样的双射呢？

搜索！

所谓状态，到底是什么？

我们以下面这题为例。

[NOI2022] 移除石子

有 n 堆石子排成一行，第 i 堆初始有 a_i 枚。你可以任意执行以下操作：

- 从某一堆中移除至少 2 枚石子；
- 选择长度至少为 3 的连续区间，从其中每堆移除 1 枚石子。

若能将所有石子全部移除，则称该局面可胜。游戏开始前，你必须将额外的 k 枚石子任意放入若干堆中。

对每个 i ，初始石子数 a_i 可任取 $[l_i, r_i]$ 中的整数。求有多少种初始局面，使得存在一种放置这 k 枚石子的方式令局面可胜。答案对 $10^9 + 7$ 取模。

$$T \leq 10, 3 \leq n \leq 1000, 0 \leq l_i \leq r_i \leq 10^9, 0 \leq k \leq 100.$$

[NOI2022] 移除石子

这类问题，我们首先考虑如何写一个 check。

[NOI2022] 移除石子

这类问题，我们首先考虑如何写一个 check。

首先，显然只有长度为 $3, 4, 5$ 的第二类操作有用，并且在一个位置连放两个相同长度的区间一定不优。所以如果确定了 k 个石子放哪，由于我们只需要考虑往后距离为 $1, 2$ 的位置，至少被覆盖了多少次，这样的状态数是很少的。

[NOI2022] 移除石子

这类问题，我们首先考虑如何写一个 check。

首先，显然只有长度为 3, 4, 5 的第二类操作有用，并且在一个位置连放两个相同长度的区间一定不优。所以如果确定了 k 个石子放哪，由于我们只需要考虑往后距离为 1, 2 的位置，至少被覆盖了多少次，这样的状态数是很少的。

然后我们加回 k ，你发现大部分情况是有单调性的，也就是说如果 k 可行，那么 $k+1$ 也可行。具体分析一下，我们发现大部分情况一定是满足这个性质的，唯一的例外是 $n \leq 3$ 。也就是说我们可以把 0/1 的 dp 改写成求最小可行的 k ，那么我们会 check 了。

[NOI2022] 移除石子

然后考虑怎么计数，假设我们总共在 check 中有 m 个状态，如果把他们全压下来则是 n^m ，这显然太离谱了。

[NOI2022] 移除石子

然后考虑怎么计数，假设我们总共在 check 中有 m 个状态，如果把他们全压下来则是 n^m ，这显然太离谱了。

我们考虑减小 m 的取值，考虑二元组 (j, k) 即后面两个位置被覆盖的次数。注意到一个位置出发的区间数量一定 ≤ 2 ，所以我们可以得到 $j \leq 4, k \leq 2$ 。

[NOI2022] 移除石子

然后考虑怎么计数，假设我们总共在 check 中有 m 个状态，如果把他们全压下来则是 n^m ，这显然太离谱了。

我们考虑减小 m 的取值，考虑二元组 (j, k) 即后面两个位置被覆盖的次数。注意到一个位置出发的区间数量一定 ≤ 2 ，所以我们可以得到 $j \leq 4, k \leq 2$ 。

这个数量已经较少了，同时我们知道，同一个位置的不同状态，他们的差值显然不会差太多，所以总状态数事实上应该是 $n \times f(m)$ ，这里 $f(m)$ 是一个关于 m 的函数。

[NOI2022] 移除石子

然后考虑怎么计数，假设我们总共在 check 中有 m 个状态，如果把他们全压下来则是 n^m ，这显然太离谱了。

我们考虑减小 m 的取值，考虑二元组 (j, k) 即后面两个位置被覆盖的次数。注意到一个位置出发的区间数量一定 ≤ 2 ，所以我们可以得到 $j \leq 4, k \leq 2$ 。

这个数量已经较少了，同时我们知道，同一个位置的不同状态，他们的差值显然不会差太多，所以总状态数事实上应该是 $n \times f(m)$ ，这里 $f(m)$ 是一个关于 m 的函数。

经过搜索，在本题限制下，这个总状态数不超过 10^4 ，于是可以通过。

先写出 check, 再 dp

上面这个题虽然比较难，但是一个很好的 dp of dp 练手题。

先写出 check, 再 dp

上面这个题虽然比较难, 但是一个很好的 dp of dp 练手题。

所谓 dp of dp, 本质就是对符合一个性质的东西计数。而符合这个性质的判定, 需要使用贪心, dp, 或者其他算法来实现。

先写出 check, 再 dp

上面这个题虽然比较难, 但是一个很好的 dp of dp 练手题。

所谓 dp of dp, 本质就是对符合一个性质的东西计数。而符合这个性质的判定, 需要使用贪心, dp, 或者其他算法来实现。

于是对这类问题的常见处理方法就是先写一个 check, 然后把 check 中所有遇到的状态“压缩”起来, 放入第二层的 dp。

先写出 check, 再 dp

上面这个题虽然比较难，但是一个很好的 dp of dp 练手题。

所谓 dp of dp，本质就是对符合一个性质的东西计数。而符合这个性质的判定，需要使用贪心，dp，或者其他算法来实现。

于是对这类问题的常见处理方法就是先写一个 check，然后把 check 中所有遇到的状态“压缩”起来，放入第二层的 dp。

下面我们再来做几道这样的题。

[AWTF2026 B] Window Records

定义长度为 n 的 $x_1, x_2, \dots, x_n$ 是合法的, 当且仅当 $x_i \leq a_i$ 且存在长度为 $2n - 1$ 的排列 $p_1, p_2, \dots, p_{2n-1}$, 满足:

对 $\forall 1 \leq t \leq n$, $p_t, p_{t+1}, \dots, p_{t+n-1}$ 的前缀最大值数量为 x_t 。
计数, $n \leq 50$ 。

[AWTF2026 B] Window Records

肯定要倒着做，考虑怎么 check：那么这个过程相当于每次 pop 一段前缀，然后给每个值加一，最后在最前面插入一个 1，最后把 $> n$ 的部分丢掉。

[AWTF2026 B] Window Records

肯定要倒着做，考虑怎么 check：那么这个过程相当于每次 pop 一段前缀，然后给每个值加一，最后在最前面插入一个 1，最后把 $> n$ 的部分丢掉。

考虑怎么快速 check，容易发现我们能扔 $> n$ 的一定扔，因为前面的东西一定更灵活。而后面加进来的东西是一定不会 pop 出去的。那么这样单次 check 就封闭了。

[AWTF2026 B] Window Records

肯定要倒着做，考虑怎么 check：那么这个过程相当于每次 pop 一段前缀，然后给每个值加一，最后在最前面插入一个 1，最后把 $> n$ 的部分丢掉。

考虑怎么快速 check，容易发现我们能扔 $> n$ 的一定扔，因为前面的东西一定更灵活。而后面加进来的东西是一定不会 pop 出去的。那么这样单次 check 就封闭了。

具体来说，限制形如：三元组 (x, y, z) 表示：当前有 x 个是后来加进来的， y 个是之前存在的，这 y 个东西必须都 $\geq z$ 。

[AWTF2026 B] Window Records

肯定要倒着做，考虑怎么 check：那么这个过程相当于每次 pop 一段前缀，然后给每个值加一，最后在最前面插入一个 1，最后把 $> n$ 的部分丢掉。

考虑怎么快速 check，容易发现我们能扔 $> n$ 的一定扔，因为前面的东西一定更灵活。而后面加进来的东西是一定不会 pop 出去的。那么这样单次 check 就封闭了。

具体来说，限制形如：三元组 (x, y, z) 表示：当前有 x 个是后来加进来的， y 个是之前存在的，这 y 个东西必须都 $\geq z$ 。

直接暴力枚举 x_t 转移就是 $O(n^5)$ 的，可以轻松通过。

[AGC067B] Modification

有一个长度为 n 的序列 $a_1, a_2, \dots, a_n$, 初始均为 0 ; 以及 m 个可行的覆盖区间 $[l_1, r_1], [l_2, r_2], \dots, [l_m, r_m]$ 。

接下来你每次会选择一个可行的覆盖区间 $[l_i, r_i]$ 和一个 $[1, k]$ 内的整数 c , 并将 $a_{l_i}, a_{l_i+1}, \dots, a_{r_i}$ 修改为 c 。

你需要选择每个区间恰好一次, 问有多少种最终可能的 a 。
答案对 998244353 取模。

$$1 \leq n \leq 100, 1 \leq m \leq \frac{n(n+1)}{2}, 1 \leq k < 998244353.$$

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

基于这个 check，发现计算什么东西能被删完是困难的，但计算不能被删完并不是如此。

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

基于这个 check，发现计算什么东西能被删完是困难的，但计算不能被删完并不是如此。

具体来说，对于所有未被删除的位置，要求所有仍然有交的区间在上面的颜色数量至少为 2。这是一个十分经典的问题。

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

基于这个 check，发现计算什么东西能被删完是困难的，但计算不能被删完并不是如此。

具体来说，对于所有未被删除的位置，要求所有仍然有交的区间在上面的颜色数量至少为 2。这是一个十分经典的问题。

而被删除的位置会形成若干极长段，段内互相独立。

[AGC067B] Modification

首先考虑怎么 check，发现我们可以按照时间从后往前能删就删。

基于这个 check，发现计算什么东西能被删完是困难的，但计算不能被删完并不是如此。

具体来说，对于所有未被删除的位置，要求所有仍然有交的区间在上面的颜色数量至少为 2。这是一个十分经典的问题。

而被删除的位置会形成若干极长段，段内互相独立。

于是可以简单 dp，时间复杂度 $O(n^4)$ 。

[AGC049E] Increment Decrement

对于一个初始序列 $a_1, a_2, \dots, a_n$, 你可以以 1 的代价对任一点 $+1/-1$, 或以 c 的代价对任一区间 $+1, -1$, 你的目标是让所有数变成 0。

现在给定 $S_1, S_2, \dots, S_n$ 。对于初始序列, 其元素 a_i 可以从 S_i 中任意选取。问所有初始序列的选取下达成目标的最小代价之和。答案对 $10^9 + 7$ 取模。

$1 \leq c \leq n \leq 50, 1 \leq |S_i| \leq 50$ 。 S_i 的所有元素都在 $[1, 10^9]$ 内。

[AGC049E] Increment Decrement

考虑给定序列怎么做，一个想法是差分。

[AGC049E] Increment Decrement

考虑给定序列怎么做，一个想法是差分。

那么问题就变成了，我们可以选两个位置 i, j ，一个加一，一个减一，代价为 $\min(c, |i - j|)$ 。显然可以用调整法证明，匹配一定形如括号匹配最优。

[AGC049E] Increment Decrement

考虑给定序列怎么做，一个想法是差分。

那么问题就变成了，我们可以选两个位置 i, j ，一个加一，一个减一，代价为 $\min(c, |i - j|)$ 。显然可以用调整法证明，匹配一定形如括号匹配最优。

但你发现这样做就没救了！现在状态数仍然记不下，因为差分可能很大。不过你发现减法一次只会减一，所以这个过程是一层一层的。我们把同一层的一起做，这样就可以分出 $O(n^2)$ 个本质不同的层。

[AGC049E] Increment Decrement

考虑给定序列怎么做，一个想法是差分。

那么问题就变成了，我们可以选两个位置 i, j ，一个加一，一个减一，代价为 $\min(c, |i - j|)$ 。显然可以用调整法证明，匹配一定形如括号匹配最优。

但你发现这样做就没救了！现在状态数仍然记不下，因为差分可能很大。不过你发现减法一次只会减一，所以这个过程是一层一层的。我们把同一层的一起做，这样就可以分出 $O(n^2)$ 个本质不同的层。

你注意到这样的话，一个前缀只有两种状态：使用第一种消除完了；和使用第二种延伸。这就是一个可以直接 dp of dp 的问题了。于是总的复杂度是 $O(n^5)$ 。

end of this part

上面的内容可能确实比较困难，但是我个人认为这是很有价值的，所以多讲了一些题。

end of this part

上面的内容可能确实比较困难，但是我个人认为这是很有价值的，所以多讲了一些题。

在算法竞赛中，这样先想怎么 check 的思想是重要的，这类题数不胜数。

end of this part

上面的内容可能确实比较困难，但是我认为这是很有价值的，所以多讲了一些题。

在算法竞赛中，这样先想怎么 check 的思想是重要的，这类题数不胜数。

如果你能熟练掌握上面这些题怎么做，你应该在这方面就不会有太大问题了。应该可以较为轻松地通过如 NOI2026D2T3 这种问题。而事实也证明，如果你能通过这个题，你是大概率能进集训队的。

end of this part

上面的内容可能确实比较困难，但是我个人认为这是很有价值的，所以多讲了一些题。

在算法竞赛中，这样先想怎么 check 的思想是重要的，这类题数不胜数。

如果你能熟练掌握上面这些题怎么做，你应该在这方面就不会有太大问题了。应该可以较为轻松地通过如 NOI2026D2T3 这种问题。而事实也证明，如果你能通过这个题，你是大概率能进集训队的。

anyway，接下来我们还是要讲今天其他的内容。

[Luogu10959] 月之谜

这道题给大家缓一缓，恢复一下信心。

求：区间 $[L, R]$ 中有多少数可以被其各位数字和整除。

保证 $0 \leq L \leq R < 2^{31}$.

[Luogu10959] 月之谜

直接数位 dp，问题在于数位和和数字本身都在变，不好处理。

[Luogu10959] 月之谜

直接数位 dp，问题在于数位和和数字本身都在变，不好处理。

但注意到数位和上界是 $9 \cdot \log R$ ，这是很小的。

[Luogu10959] 月之谜

直接数位 dp，问题在于数位和和数字本身都在变，不好处理。

但注意到数位和上界是 $9 \cdot \log R$ ，这是很小的。

所以我们可以枚举数位和 s ，在记忆化搜索结束的时候，只有当前数位和和我们枚举的 s 相同时，再记录答案。

[Luogu10959] 月之谜

直接数位 dp，问题在于数位和和数字本身都在变，不好处理。

但注意到数位和上界是 $9 \cdot \log R$ ，这是很小的。

所以我们可以枚举数位和 s ，在记忆化搜索结束的时候，只有当前数位和和我们枚举的 s 相同时，再记录答案。

时间复杂度 $O(\log^3 R)$ 。

一道我不知道原题是什么的题

定义函数 $f(x)$ 为 x 的各位数字之和, 例如
 $f(13) = 4, f(233) = 8$ 。

给定正整数 n 和 s , 若 $\sum_{i=l}^r f(i) \leq s$, 则区间 $[l, r]$ 是勃勃生机的。

问满足 $0 \leq l \leq r \leq n$ 的勃勃生机区间有多少。

$$n \leq 10^{18}, s \leq 10^6。$$

一道我不知道原题是什么的题

容易发现满足小于等于 s 的区间的长度一定不超过 10^5 ，于是我们将 n 按 10^5 分块。

一道我不知道原题是什么的题

容易发现满足小于等于 s 的区间的长度一定不超过 10^5 ，于是我们将 n 按 10^5 分块。

有三种类型：

一是区间在整块内，我们只需要使用数位 dp，求出 $\frac{n}{10^5}$ 的不同的数位之和各有几种，然后双指针计算；

一道我不知道原题是什么的题

容易发现满足小于等于 s 的区间的长度一定不超过 10^5 ，于是我们将 n 按 10^5 分块。

有三种类型：

一是区间在整块内，我们只需要使用数位 dp，求出 $\frac{n}{10^5}$ 的不同的数位之和各有几种，然后双指针计算；

二是区间两端点在相邻的两个整块内，求出 $\frac{n}{10^5}$ 不同的数位之和各有几种，以及进位后会发生怎样的改变，这个只要额外在数位 dp 时记录末尾有几个连续的 9；

一道我不知道原题是什么的题

容易发现满足小于等于 s 的区间的长度一定不超过 10^5 ，于是我们将 n 按 10^5 分块。

有三种类型：

一是区间在整块内，我们只需要使用数位 dp，求出 $\frac{n}{10^5}$ 的不同的数位之和各有几种，然后双指针计算；

二是区间两端点在相邻的两个整块内，求出 $\frac{n}{10^5}$ 不同的数位之和各有几种，以及进位后会发生怎样的改变，这个只要额外在数位 dp 时记录末尾有几个连续的 9；

三是有部分在散块内，暴力直接算。

数据结构优化 dp

dp 的时候，我们很多时候会尝试使用数据结构优化转移的代价。

数据结构优化 dp

dp 的时候，我们很多时候会尝试使用数据结构优化转移的代价。

常见情况有两种，一是一个状态有很多可能的前驱，我们要用数据结构求出其中最优的那个；

数据结构优化 dp

dp 的时候，我们很多时候会尝试使用数据结构优化转移的代价。

常见情况有两种，一是一个状态有很多可能的前驱，我们要用数据结构求出其中最优的那个；

二是有很多状态需要同时转移，这时我们使用数据结构维护所有状态从而快速转移。

[NOIP2023] 天天爱打卡

试运行持续 n 天。每天可以选择跑步或休息；每跑步一天，能量减少 d ，且不能连续跑步超过 k 天。初始能量为 0，过程中能量可以为负。

共有 m 个挑战。第 i 个挑战为 (x_i, y_i, v_i) ：若第 $x_i - y_i + 1$ 天至第 x_i 天均跑步，则在第 x_i 天获得 v_i 点能量。

求第 n 天结束后可能达到的最大能量。

$1 \leq t \leq 10, 1 \leq k \leq n \leq 10^9, 1 \leq m \leq 10^5, 1 \leq y_i \leq x_i \leq n, 1 \leq d, v_i \leq 10^9$ 。

[NOIP2023] 天天爱打卡

按时间 dp，设 $f(i, j)$ 表示考虑前 i 天，上一次休息是第 j 天，那么最大能量是多少。

[NOIP2023] 天天爱打卡

按时间 dp，设 $f(i, j)$ 表示考虑前 i 天，上一次休息是第 j 天，那么最大能量是多少。

转移分为几步：首先是第 i 天休息，那么就是 $f(i, i) \leftarrow \max f(i-1, j)$ 。

[NOIP2023] 天天爱打卡

按时间 dp，设 $f(i, j)$ 表示考虑前 i 天，上一次休息是第 j 天，那么最大能量是多少。

转移分为几步：首先是第 i 天休息，那么就是 $f(i, i) \leftarrow \max f(i-1, j)$ 。

然后对于继续跑步，首先 $f(i, j) \leftarrow f(i-1, j) - d$ ，然后对于 $x_p = i$ 的挑战，将 $\forall j \leq x_p - y_p$ 的 $f(i, j)$ 都加 v_p 。最后再把 $f(i, i-k)$ 设为 $-\infty$ 。

[NOIP2023] 天天爱打卡

按时间 dp，设 $f(i, j)$ 表示考虑前 i 天，上一次休息是第 j 天，那么最大能量是多少。

转移分为几步：首先是第 i 天休息，那么就是 $f(i, i) \leftarrow \max f(i-1, j)$ 。

然后对于继续跑步，首先 $f(i, j) \leftarrow f(i-1, j) - d$ ，然后对于 $x_p = i$ 的挑战，将 $\forall j \leq x_p - y_p$ 的 $f(i, j)$ 都加 v_p 。最后再把 $f(i, i-k)$ 设为 $-\infty$ 。

这些都可以直接使用线段树维护，时间复杂度 $O(n \log n)$ 。

[CF671D] Roads in Yusland

Yusland 的 n 个路口由 $n - 1$ 条双向道路连接成一棵树，根为路口 1。

有 m 名工人。雇佣第 i 名工人需支付 c_i ，他可以修复 u_i 到其祖先 v_i 的路径上的所有道路。

选择若干名工人，使树上的每条道路都至少被修复一次，并最小化总费用。若无法修复所有道路，输出 -1 。

$1 \leq n, m \leq 3 \times 10^5$, $1 \leq c_i \leq 10^9$, 且 v_i 是 u_i 的祖先。

[CF671D] Roads in Yusland

记 $f(i, j)$ 表示, 以 i 为根的子树全覆盖, 并且子树内还选择了一个伸到深度为 j 的祖先的路径, 最小代价为多少。

[CF671D] Roads in Yusland

记 $f(i, j)$ 表示, 以 i 为根的子树全覆盖, 并且子树内还选择了一个伸到深度为 j 的祖先的路径, 最小代价为多少。

转移则是 $f(u, i) = \min_{v \in \text{son}_u} [f(v, i) + \sum_{w \neq v} \min_j f(w, j)]$ 。

[CF671D] Roads in Yusland

记 $f(i, j)$ 表示, 以 i 为根的子树全覆盖, 并且子树内还选择了一个伸到深度为 j 的祖先的路径, 最小代价为多少。

转移则是 $f(u, i) = \min_{v \in \text{son}_u} [f(v, i) + \sum_{w \neq v} \min_j f(w, j)]$ 。

记 $g(u, i) = f(u, i) - \min_j f(u, j)$, 那么转移形如 $g(u, i) = S + \min_v g(v, i)$, 所以我们的问题就是按位取 $\min$ 。

[CF671D] Roads in Yusland

记 $f(i, j)$ 表示, 以 i 为根的子树全覆盖, 并且子树内还选择了一个伸到深度为 j 的祖先的路径, 最小代价为多少。

转移则是 $f(u, i) = \min_{v \in \text{son}_u} [f(v, i) + \sum_{w \neq v} \min_j f(w, j)]$ 。

记 $g(u, i) = f(u, i) - \min_j f(u, j)$, 那么转移形如 $g(u, i) = S + \min_v g(v, i)$, 所以我们的问题就是按位取 $\min$ 。

可以使用线段树合并优化, 时空复杂度均为 $O(n \log n)$ 。

[CF1129D] Isolation

给定长度为 n 的整数数组 a 。

将数组划分为若干个不相交的非空连续段。要求每一段中，**恰好出现一次的不同整数不超过 k 个**。

求满足条件的划分方案数，答案对 998244353 取模。

$$1 \leq k \leq n \leq 10^5, 1 \leq a_i \leq n。$$

[CF1129D] Isolation

自然有一个最基础的 dp, 就是 $f(i) = \sum_{[j,i] \text{ is legal}} f(j-1)$ 。

[CF1129D] Isolation

自然有一个最基础的 dp，就是 $f(i) = \sum_{[j,i] \text{ is legal}} f(j-1)$ 。

我们需要考虑的就是，从 i 到 $i+1$ 时，哪些 j 对应的 $f(j-1)$ 不合法了。换句话说，哪些 $[j, i+1]$ 从 $\leq k$ 变成 $> k$ 了。

[CF1129D] Isolation

自然有一个最基础的 dp, 就是 $f(i) = \sum_{[j,i] \text{ is legal}} f(j-1)$ 。

我们需要考虑的就是, 从 i 到 $i+1$ 时, 哪些 j 对应的 $f(j-1)$ 不合法了。换句话说, 哪些 $[j, i+1]$ 从 $\leq k$ 变成 $> k$ 了。

这就要求原先 $[j, i]$ 的出现次数恰好是 k , 且新增的数出现恰好一次。我们设 pre_i 表示上个和 i 相同的数, 那么要求 $pre_i < j < i$ 。

[CF1129D] Isolation

现在问题抽象为，有两个序列 a, b 。每次对 b 区间加，然后你要查询一个区间里，满足 $b_i = k$ 的所有 i 的 a_i 的和。

[CF1129D] Isolation

现在问题抽象为，有两个序列 a, b 。每次对 b 区间加，然后你要查询一个区间里，满足 $b_i = k$ 的所有 i 的 a_i 的和。

这只需要分块就行了，散块暴力，整块开个桶。

[CF1129D] Isolation

现在问题抽象为，有两个序列 a, b 。每次对 b 区间加，然后你要查询一个区间里，满足 $b_i = k$ 的所有 i 的 a_i 的和。

这只需要分块就行了，散块暴力，整块开个桶。

时间复杂度 $O(n\sqrt{n})$ 。

[CSP-S 2022] 数据传输

给定一棵 n 个结点的树，第 i 个结点的处理时间为 v_i 。若两结点间的树上距离不超过 k ，则可以直接传输数据。

对于一次请求 (s, t) ，选择结点序列 $c_1 = s, c_2, \dots, c_m = t$ ，要求相邻结点间距离均不超过 k 。传输代价为 $\sum_{i=1}^m v_{c_i}$ 。

对每次请求，求从 s 向 t 传输数据的最小代价。

$$1 \leq n, Q \leq 2 \times 10^5, 1 \leq k \leq 3, 1 \leq v_i \leq 10^9。$$

[CSP-S 2022] 数据传输

首先我们把这条树链提出来，分析性质。考虑怎么 dp，可能的问题是我们跳到树链外，然后跳回来。

[CSP-S 2022] 数据传输

首先我们把这条树链提出来，分析性质。考虑怎么 dp，可能的问题是我们跳到树链外，然后跳回来。

于是我们给所有点分类，即，给每个点找到其在树链上的最近点。

[CSP-S 2022] 数据传输

首先我们把这条树链提出来，分析性质。考虑怎么 dp，可能的问题是我们跳到树链外，然后跳回来。

于是我们给所有点分类，即，给每个点找到其在树链上的最近点。

现在就是一个可以 dp 的问题了，设 $f(i, j)$ ，表示跳到 i 所在的这一类，上一个点离 i 距离为 j 。你可能会有疑问它会不会先跳到后面的类别再回到前面，但事实证明这是不会的。

[CSP-S 2022] 数据传输

首先我们把这条树链提出来，分析性质。考虑怎么 dp，可能的问题是我们跳到树链外，然后跳回来。

于是我们给所有点分类，即，给每个点找到其在树链上的最近点。

现在就是一个可以 dp 的问题了，设 $f(i, j)$ ，表示跳到 i 所在的这一类，上一个点离 i 距离为 j 。你可能会有疑问它会不会先跳到后面的类别再回到前面，但事实证明这是不会的。

你注意到，对于树上的一条边 (u, v) ，我们永远知道每个 $f(u, i)$ 到 $f(v, j)$ 的贡献，这是可以在一开始预处理的。

[CSP-S 2022] 数据传输

首先我们把这条树链提出来，分析性质。考虑怎么 dp，可能的问题是我们跳到树链外，然后跳回来。

于是我们给所有点分类，即，给每个点找到其在树链上的最近点。

现在就是一个可以 dp 的问题了，设 $f(i, j)$ ，表示跳到 i 所在的这一类，上一个点离 i 距离为 j 。你可能会有疑问它会不会先跳到后面的类别再回到前面，但事实证明这是不会的。

你注意到，对于树上的一条边 (u, v) ，我们永远知道每个 $f(u, i)$ 到 $f(v, j)$ 的贡献，这是可以在一开始预处理的。

而转移是 $\max/+$ 卷积，具有矩阵的性质。所以我们通过树剖 + 线段树维护区间矩阵除法的结果，时间复杂度 $O(n \log n)$ 。

斜率优化

所有的斜率优化问题都可以看作这样的问题：有 n 个点 (x_i, y_i) 。现在有一个斜率为 k 的直线，从无穷高处向下滑，问第一次接触的位置。

斜率优化

所有的斜率优化问题都可以看作这样的问题：有 n 个点 (x_i, y_i) 。现在有一个斜率为 k 的直线，从无穷高处向下滑，问第一次接触的位置。

做法是求出上凸壳，然后二分出斜率的分界，就是第一个交点的位置。

[CF1083E] The Fair Nut and Rectangles

给定 n 个左下角为 $(0, 0)$ 、右上角为 (x_i, y_i) 的矩形，第 i 个矩形还有代价 a_i 。

选择若干个矩形，其收益定义为所选矩形并集的面积减去这些矩形的代价之和。

求最大可能收益。保证不存在一个矩形包含另一个矩形。

$$1 \leq n \leq 10^6, 1 \leq x_i, y_i \leq 10^9, 0 \leq a_i \leq x_i y_i.$$

[CF1083E] The Fair Nut and Rectangles

假设所有东西是按 x_i 升序的，那么假定我们选择了一个集合 S 为选择的矩形，那么并集大小就是 $\sum_{i=1}^{|S|} y_{S_i} (x_{S_i} - x_{S_{i-1}})$.

[CF1083E] The Fair Nut and Rectangles

假设所有东西是按 x_i 升序的，那么假定我们选择了一个集合 S 为选择的矩形，那么并集大小就是 $\sum_{i=1}^{|S|} y_{S_i} (x_{S_i} - x_{S_{i-1}})$.

于是可以 dp ，设 $f(i)$ 表示选了第 i 个，转移是 $f(i) = -a_i + \max_{0 \leq j < i} y_i (x_i - x_j) + f_j$.

[CF1083E] The Fair Nut and Rectangles

假设所有东西是按 x_i 升序的，那么假定我们选择了一个集合 S 为选择的矩形，那么并集大小就是 $\sum_{i=1}^{|S|} y_{S_i} (x_{S_i} - x_{S_{i-1}})$ 。

于是可以 dp ，设 $f(i)$ 表示选了第 i 个，转移是 $f(i) = -a_i + \max_{0 \leq j < i} y_i (x_i - x_j) + f_j$ 。

注意到这是一个斜率优化的形式，第 j 个点对应的二维平面上的点是 (x_j, f_j) ，斜率是 y_i 。

[CF1083E] The Fair Nut and Rectangles

假设所有东西是按 x_i 升序的，那么假定我们选择了一个集合 S 为选择的矩形，那么并集大小就是 $\sum_{i=1}^{|S|} y_{S_i}(x_{S_i} - x_{S_{i-1}})$ 。

于是可以 dp ，设 $f(i)$ 表示选了第 i 个，转移是 $f(i) = -a_i + \max_{0 \leq j < i} y_i(x_i - x_j) + f_j$ 。

注意到这是一个斜率优化的形式，第 j 个点对应的二维平面上的点是 (x_j, f_j) ，斜率是 y_i 。

由于 x_i 递增， y_i 递减，我们可以使用单调队列维护当前的凸壳。这样复杂度除了排序以外，就是严格线性的。

维护凸壳太弱了，你发现如果点加入的顺序是随机的，我们就只能使用平衡树维护凸壳了。

维护凸壳太弱了，你发现如果点加入的顺序是随机的，我们就只能使用平衡树维护凸壳了。

所以有没有更好的方法？

李超树

维护凸壳太弱了，你发现如果点加入的顺序是随机的，我们就只能使用平衡树维护凸壳了。

所以有没有更好的方法？

李超树！具体来说，它和线段树的结构差不多。但每个节点维护的是在当前节点的中点处，最优的函数是谁。

[QOJ8007] Egg Drop Challenge

有 n 个人分别位于高度 $h_1 < h_2 < \dots < h_n$, 第 n 个人初始拿着一个鸡蛋。

第 i 个人拿到鸡蛋后, 可以立即以 $[0, v_i]$ 内任意实数速度向下抛出; 鸡蛋经过第 i 个人时, 若速度不超过 u_i , 他可以将其接住并使其停止。鸡蛋下落加速度为 1, 忽略空气阻力, 接住和抛出均不耗时。

可以选择任意若干人中转。求使鸡蛋最终被第 1 个人安全接住的最短时间; 若无法做到, 输出 -1 。

$$2 \leq n \leq 3 \times 10^5, 1 \leq h_i \leq 10^{18}, 1 \leq v_i, u_i \leq 10^9。$$

[QOJ8007] Egg Drop Challenge

首先有一个暴力 dp, 记 $f(i)$ 表示第 i 个人开始往下扔, 最短时间是多少。

[QOJ8007] Egg Drop Challenge

首先有一个暴力 dp，记 $f(i)$ 表示第 i 个人开始往下扔，最短时间是多少。

转移是 $f(i) = \min_{j < i} f(j) + \text{cost}(j, i)$ 。其中 $\text{cost}(j, i)$ 需要分类讨论：

[QOJ8007] Egg Drop Challenge

首先有一个暴力 dp，记 $f(i)$ 表示第 i 个人开始往下扔，最短时间是多少。

转移是 $f(i) = \min_{j < i} f(j) + \text{cost}(j, i)$ 。其中 $\text{cost}(j, i)$ 需要分类讨论：

如果 $(v_i + u_j)(u_j - v_i) \geq 2(h_i - h_j)$ ，那么用时为 $-v_i + \sqrt{v_i^2 + 2(h_i - h_j)}$ ，否则为 $u_j - \sqrt{u_j^2 - 2(h_i - h_j)}$ 。

[QOJ8007] Egg Drop Challenge

首先有一个暴力 dp, 记 $f(i)$ 表示第 i 个人开始往下扔, 最短时间是多少。

转移是 $f(i) = \min_{j < i} f(j) + \text{cost}(j, i)$ 。其中 $\text{cost}(j, i)$ 需要分类讨论:

如果 $(v_i + u_j)(u_j - v_i) \geq 2(h_i - h_j)$, 那么用时为 $-v_i + \sqrt{v_i^2 + 2(h_i - h_j)}$, 否则为 $u_j - \sqrt{u_j^2 - 2(h_i - h_j)}$ 。

这两个结构都相当于我们有一个自变量 x , 要求约束条件下若干函数 $f_i(x) = a_i + \sqrt{b_i + x}$ 的最值。

[QOJ8007] Egg Drop Challenge

考虑两个函数 $f(x) = a + \sqrt{b + x}$, $g(x) = c + \sqrt{d + x}$, 容易发现只有一个交点, 也就是说, 满足李超树的限制!

[QOJ8007] Egg Drop Challenge

考虑两个函数 $f(x) = a + \sqrt{b + x}$, $g(x) = c + \sqrt{d + x}$, 容易发现只有一个交点, 也就是说, 满足李超树的限制!

约束等价于比较 $2h_j + u_j^2$ 和 $2h_i + v_i^2$, 所以这是好处理的, 我们只需要在外侧再开个数据结构, 比方说树状数组, 就可以使用树套树完成我们的需求。

[QOJ8007] Egg Drop Challenge

考虑两个函数 $f(x) = a + \sqrt{b + x}$, $g(x) = c + \sqrt{d + x}$, 容易发现只有一个交点, 也就是说, 满足李超树的限制!

约束等价于比较 $2h_j + u_j^2$ 和 $2h_i + v_i^2$, 所以这是好处理的, 我们只需要在外侧再开个数据结构, 比方说树状数组, 就可以使用树套树完成我们的需求。

时间复杂度为 $O(n \log^2 n)$, 可以通过。

凸优化 dp

当你的 dp 数组 $f(u, i)$ 是一个关于 i 的凸函数的时候，其拥有着许多方便的维护方式。

凸优化 dp

当你的 dp 数组 $f(u, i)$ 是一个关于 i 的凸函数的时候，其拥有着许多方便的维护方式。

其中一种是 slope trick，我们以下面这题为例，来介绍一下。

[ARC123D] Inc, Dec - Decomposition

给定整数序列 $A = (A_1, A_2, \dots, A_N)$ 。

构造两个整数序列 B, C , 满足 $A_i = B_i + C_i$, 且 B 单调不降、 C 单调不升。

求 $\sum_{i=1}^N (|B_i| + |C_i|)$ 的最小值。

$1 \leq N \leq 2 \times 10^5$, $-10^8 \leq A_i \leq 10^8$ 。

[ARC123D] Inc, Dec - Decomposition

考虑一个最暴力的东西，设 $f(i, x)$ 表示， $b_i = x$ 的基础下，最小值是多少。

[ARC123D] Inc, Dec - Decomposition

考虑一个最暴力的东西，设 $f(i, x)$ 表示， $b_i = x$ 的基础下，最小值是多少。

那么转移是一个前缀 checkmin ，然后再加一个凸函数。所以我们可以归纳证明其仍然是凸的。

[ARC123D] Inc, Dec - Decomposition

考虑一个最暴力的东西，设 $f(i, x)$ 表示， $b_i = x$ 的基础下，最小值是多少。

那么转移是一个前缀 checkmin ，然后再加一个凸函数。所以我们可以归纳证明其仍然是凸的。

注意到，凸函数的二阶差分一定非负，而我们只需要前两个位置的值和二阶差分的值，就可以还原整个函数。

[ARC123D] Inc, Dec - Decomposition

考虑一个最暴力的东西，设 $f(i, x)$ 表示， $b_i = x$ 的基础下，最小值是多少。

那么转移是一个前缀 checkmin ，然后再加一个凸函数。所以我们可以归纳证明其仍然是凸的。

注意到，凸函数的二阶差分一定非负，而我们只需要前两个位置的值和二阶差分的值，就可以还原整个函数。

而求差分是线性的，所以相当于直接合并两个二阶差分，可以使用堆来维护，时间复杂度 $O(n \log n)$ 。

[APIO2016] 烟火表演

给定一棵以结点 1 为根的树。结点 $1, \dots, N$ 为分叉点，结点 $N+1, \dots, N+M$ 为烟花，每条边有一个非负整数长度。

烟花的爆炸时间等于根到该烟花路径上的边长之和。你可以修改任意边的长度，修改代价为新旧长度之差的绝对值，且修改后边长不能为负。

求使所有烟花 **同时爆炸** 的最小总代价。

$$1 \leq N + M \leq 3 \times 10^5。$$

[APIO2016] 烟火表演

设 $f(i, x)$ 表示以 i 为根的子树，所有叶子到 i 的距离为 x 的最小代价。

[APIO2016] 烟火表演

设 $f(i, x)$ 表示以 i 为根的子树，所有叶子到 i 的距离为 x 的最小代价。

转移我们只需要考虑一条祖先边的影响，因为不同子树的合并事实上就是按位相加。

[APIO2016] 烟火表演

设 $f(i, x)$ 表示以 i 为根的子树，所有叶子到 i 的距离为 x 的最小代价。

转移我们只需要考虑一条祖先边的影响，因为不同子树的合并事实上就是按位相加。

猜测这个函数是凸的，然后观察如果假设成立，合并会是怎样的：

首先最小值会往右平移边长位，然后把右边全部 pop 掉，插入一条斜率为 1 的直线，再在左边插入斜率为 -1 ，长度为边长的直线。

[APIO2016] 烟火表演

设 $f(i, x)$ 表示以 i 为根的子树，所有叶子到 i 的距离为 x 的最小代价。

转移我们只需要考虑一条祖先边的影响，因为不同子树的合并事实上就是按位相加。

猜测这个函数是凸的，然后观察如果假设成立，合并会是怎样的：

首先最小值会往右平移边长位，然后把右边全部 pop 掉，插入一条斜率为 1 的直线，再在左边插入斜率为 -1 ，长度为边长的直线。

这一切和上面那道题都是类似的，可以使用可并堆维护，时间复杂度 $O(n \log n)$ 。

Aliens' Trick 与决策单调性

然后，另一个常见的优化手法是 Aliens trick，在中国的算法竞赛界又名 wqs 二分。

Aliens' Trick 与决策单调性

然后，另一个常见的优化手法是 Aliens trick，在中国的算法竞赛界又名 wqs 二分。

同时，在许多 dp 的时候，你会发现，如果 $f(i-1)$ 对于 $f(j)$ 的贡献已经比 $f(i)$ 劣了，那么在以后永远更劣。

我们一般称这种性质为决策单调性。

Aliens' Trick 与决策单调性

然后，另一个常见的优化手法是 Aliens trick，在中国的算法竞赛界又名 wqs 二分。

同时，在许多 dp 的时候，你会发现，如果 $f(i-1)$ 对于 $f(j)$ 的贡献已经比 $f(i)$ 劣了，那么在以后永远更劣。

我们一般称这种性质为决策单调性。

下面是一道例题。

[IOI 2000] 邮局

数轴上有 n 个村庄，第 i 个村庄的坐标为 a_i 。

选择其中 m 个村庄建立邮局。每个村庄到最近邮局的距离为两者坐标之差的绝对值。

求所有村庄到最近邮局的距离之和的最小值。

$$1 \leq m \leq n \leq 5 \times 10^5, 0 \leq a_i \leq 2 \times 10^6.$$

[IOI 2000] 邮局

首先这个问题等价于选择 m 个区间覆盖 n 个点，一个区间的代价是其中所有点到一个点的距离和最小值：那么我们肯定选择中位数。所以一个区间的代价可以预处理后 $O(1)$ 查询。

于是有一个暴力 dp，时间复杂度 $O(n^3)$ 。

[IOI 2000] 邮局

首先这个问题等价于选择 m 个区间覆盖 n 个点，一个区间的代价是其中所有点到一个点的距离和最小值：那么我们肯定选择中位数。所以一个区间的代价可以预处理后 $O(1)$ 查询。

于是有一个暴力 dp，时间复杂度 $O(n^3)$ 。

你注意到贡献函数满足 **四边形不等式**，所以具有决策单调性，那么我们就可以使用决策单调性分治，优化到 $O(n^2 \log n)$ 。

[IOI 2000] 邮局

首先这个问题等价于选择 m 个区间覆盖 n 个点，一个区间的代价是其中所有点到一个点的距离和最小值：那么我们肯定选择中位数。所以一个区间的代价可以预处理后 $O(1)$ 查询。

于是有一个暴力 dp，时间复杂度 $O(n^3)$ 。

你注意到贡献函数满足 **四边形不等式**，所以具有决策单调性，那么我们就可以使用决策单调性分治，优化到 $O(n^2 \log n)$ 。

接着，我们发现答案对于 k 是一个凸函数。我们知道原问题求最值是好求的，那么我们可以变换一下问题，使得 $f(i, j)$ 的值加 $j \times C$ ，这样求出的最值就是凸壳上第一个斜率 $< C$ 的位置。这里的 C 是一个任意常数。

[IOI 2000] 邮局

既然 C 是任意的，所以我们二分它。

[IOI 2000] 邮局

既然 C 是任意的，所以我们二分它。

在 dp 的时候额外记录最值对应的 j 是什么。根据 j 与 k 的大小关系，我们可以判断当前的 C 是大了还是小了。

[IOI 2000] 邮局

既然 C 是任意的，所以我们二分它。

在 dp 的时候额外记录最值对应的 j 是什么。根据 j 与 k 的大小关系，我们可以判断当前的 C 是大了还是小了。

这样就可以做到 $O(n \log^2 n)$ ，这也就是 Aliens' Trick.

[IOI2016] Aliens

给定一个 $m \times m$ 的网格，其中有 n 个兴趣点，第 i 个点位于单元格 (r_i, c_i) ，同一单元格内可以有多个点。

一张照片必须是边与网格平行的正方形，且其一对对角顶点位于网格主对角线上。你至多拍摄 k 张照片，要求覆盖所有兴趣点。

若一个单元格被多张照片覆盖，只计算一次。求被覆盖的不同单元格数量的最小值。

$$1 \leq k \leq n \leq 10^5, 1 \leq m \leq 10^6, 0 \leq r_i, c_i < m.$$

[IOI2016] Aliens

考虑一个横坐标区域为 $[l, r]$ 的矩形，那么包含点 (r_i, c_i) 的条件为 $l \leq r_i, c_i \leq r$ ，所以这相当于包含一个线段。

[IOI2016] Aliens

考虑一个横坐标区域为 $[l, r]$ 的矩形，那么包含点 (r_i, c_i) 的条件为 $l \leq r_i, c_i \leq r$ ，所以这相当于包含一个线段。

所以现在是一个这样的问题，选不超过 k 个区间包含所有给定区间，使得区间长度平方和最小。

[IOI2016] Aliens

考虑一个横坐标区域为 $[l, r]$ 的矩形，那么包含点 (r_i, c_i) 的条件为 $l \leq r_i, c_i \leq r$ ，所以这相当于包含一个线段。

所以现在是一个这样的问题，选不超过 k 个区间包含所有给定区间，使得区间长度平方和最小。

首先有两个显然的事情，被包含的选择区间和限制区间都没有用，也就是说所有有用的区间两个端点是分别递增的。

[IOI2016] Aliens

考虑一个横坐标区域为 $[l, r]$ 的矩形，那么包含点 (r_i, c_i) 的条件为 $l \leq r_i, c_i \leq r$ ，所以这相当于包含一个线段。

所以现在是一个这样的问题，选不超过 k 个区间包含所有给定区间，使得区间长度平方和最小。

首先有两个显然的事情，被包含的选择区间和限制区间都没有用，也就是说所有有用的区间两个端点是分别递增的。

换句话说，我们考虑最终选择区间里有一个右端点为 R 的区间，那么所有右端点 $\leq R$ 的限制区间应当被所有右端点 $\leq R$ 的选择区间所包含。

[IOI2016] Aliens

所以说，设 $f(i, j)$ 表示选了一个右端点为 i 的，当前选的区域个数为 j ，那么转移就是

$$f(i, j) = \min_{k \leq i} f(k, j-1) + (i - \min_{k \leq p \leq i} l_p)^2$$

所以我们就有了 $O(n^3)$ 的做法。

[IOI2016] Aliens

所以说, 设 $f(i, j)$ 表示选了一个右端点为 i 的, 当前选的区域个数为 j , 那么转移就是

$$f(i, j) = \min_{k \leq i} f(k, j-1) + (i - \min_{k \leq p \leq i} l_p)^2$$

所以我们就有了 $O(n^3)$ 的做法。

优化考虑这玩意一看就是凸的, 使用 Aliens' Trick 去掉第二维, 那么转移形如

$$f(i) = C + \min_{k \leq i} f(k) + (i - \min_{k \leq p \leq i} l_p)^2$$

[IOI2016] Aliens

所以说，设 $f(i, j)$ 表示选了一个右端点为 i 的，当前选的区间个数为 j ，那么转移就是

$$f(i, j) = \min_{k \leq i} f(k, j-1) + (i - \min_{k \leq p \leq i} l_p)^2$$

所以我们就有了 $O(n^3)$ 的做法。

优化考虑这玩意一看就是凸的，使用 Aliens' Trick 去掉第二维，那么转移形如

$$f(i) = C + \min_{k \leq i} f(k) + (i - \min_{k \leq p \leq i} l_p)^2$$

我们使用单调栈维护 $\min_{k \leq p \leq i} l_p$ ，那么我们需要实现的东西就只有区间加，区间最大值，可以直接使用线段树维护。时间复杂度 $O(n \log n)$ 。

谢谢大家！